Document Knowledge Graph to Enhance Question Answering with Retrieval Augmented Generation

1st Simon Knollmeyer
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0009-0002-1429-6992

2nd Muhammad Uzair Akmal
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0009-0007-3961-1174

3rd Leonid Koval
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0000-0003-4845-6579

4th Saara Asif
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0009-0006-1284-5635

5th Selvine G. Mathias
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0000-0002-6549-0763

6th Daniel Großmann
AIMotion Bavaria
Technische Hochschule Ingolstadt
Ingolstadt, Germany
0000-0002-7388-5757

2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA) | 979-8-3503-6123-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ETFA61755.2024.10711054

***Abstract*—Reusing and managing existing knowledge from available documents is crucial for success in the factory planning domain. By leveraging Artificial Intelligence (AI) and Question Answering (QA) systems, users can query a document corpus through a chat-based application and receive precise answers. The recent advancements in Large Language Models (LLMs) and their linguistic capabilities present new opportunities for such applications. Utilizing the methodology of Retrieval Augmented Generation (RAG), document sections are provided to the LLM based on user queries. However, existing RAG implementations that use vector databases as document repositories face limitations when answering questions that extend beyond the text content of the documents. To address this issue, this paper proposes a concept to enhance RAG systems by integrating a Knowledge Graph (KG) constructed from the document structures.**

***Index Terms*—Information management, Retrieval Augmented Generation, Knowledge Graph, Large Language Models**

I. Introduction

Effective knowledge management is a key factor for successful and efficient task completion within the factory planning domain. The volume of available data is continually increasing due to the digitalization of business processes and the implementation of Industry 4.0 concepts and its enabling technologies. Within this domain, a substantial amount of knowledge is stored in an unstructured format across various documents. To provide employees with user-friendly chat access to this knowledge, AI methods such as document QA can be employed. Users can post text-based questions and receive answers derived from a document knowledge base [1].

One viable approach for enabling AI-supported document knowledge QA in the factory planning domain is through the use of LLM-powered chatbots. In a RAG approach, the language model facilitates text-based interaction between the user and the knowledge base, typically implemented as a vector database. This database contains the text content of documents and supplies relevant sections to the LLM based on the user query context [2]. A significant challenge here is that the vector database can only retrieve text sections from the documents, without accounting for meta-data or relationships between individual documents. Moreover, as the number of documents grows, it becomes advantageous for the functionality of the chatbot to manage the structure of the knowledge base and filter content effectively.

KGs have significant potential to address these challenges due to their focus on data relationships and the ease of customizing their schema compared to other database models. Therefore, we aim to demonstrate a general methodology for building, managing, and accessing the knowledge base within a RAG framework with the assistance of a KG.

Overall, the proposed concept addresses the following challenges by extending established RAG approaches:

- Managing document knowledge along with their meta-data within a knowledge base for RAG.
- Enabling the filtering and retrieval of content in a combined knowledge base consisting of a KG and another database.

The following sections provide an overview of related work on RAG and KGs. Building on this foundation, we introduce the concept for the structure of the KG and its integration into the RAG architecture in Section 3.

II. Related Work

This section presents related work to the considered topics. Furthermore, existing limits and challenges regarding the combination of KGs and RAG are outlined.

A. Retrieval Augmented Generation

RAG is a methodology designed to enhance the quality of information in texts generated by LLMs. This approach incorporates a search component that retrieves necessary information from a knowledge base before the text generation process. The LLM then uses this retrieved information to answer user queries. The knowledge base can include internal

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on February 09,2026 at 19:08:08 UTC from IEEE Xplore. Restrictions apply.

corporate content, allowing for chat-based interaction without integrating this content into the LLM's training data [3].

Nao et al. [2] provide a comprehensive overview of RAG in combination with LLMs, categorizing various approaches. A RAG approach for documents typically involves the following three phases:

Phase 1: Incorporation and indexing of documents
Phase 2: Retrieval of relevant contents
Phase 3: Transfer of the retrieved results to the LLM.

Common implementations of the RAG architecture are described in [4] and [5]. During the import phase, documents are divided into smaller, content-related sections known as chunks. These chunks are treated as independent objects and made available to the LLM as context. Embedding models generate vectors for each chunk, representing the semantic content numerically, and store these in a vector database. In the retrieval phase, the user's query is also transformed into a vector, which is used to retrieve the most relevant chunks from the database. In the final phase, these chunks are transferred to the LLM to generate the response.

B. Knowledge Graphs for Document Management

Within the literature, two different approaches for using KGs in document management and RAG can be identified. Based on the structure and content of the graph, the application of KGs can be essentially divided into the categories shown in Figure 1.

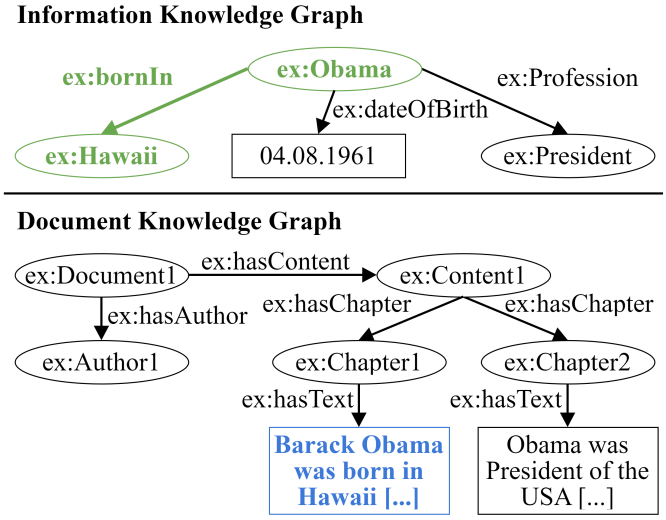

Fig. 1. Information-KG and Document-KG.

The upper part of Figure 1 shows an excerpt for an *Information-KG* (IKG) according to the categorization. The actual knowledge is represented directly by nodes and edges. For example, the question "Where was Barack Obama born?" can be answered directly by the green colored triple `(Obama, bornIn, Hawaii)` as presented in [6]. In the lower area, a part of a *Document-KG* (DKG) according to the classification is shown. The (hierarchical) relationships in documents and meta-data like the author are stored in this graph through the nodes and edges as described by [7]. To answer the aforementioned question, the text object marked in blue must be found using a search algorithm. Subsequently, further content from the DKG can be transferred to the LLM together with the text.

The two types of KGs differ in their creation process. DKGs can be created based on rules if the structure and metadata of the document are known. Creating IKGs is more complex, as unstructured information, including its semantics, needs to be transformed into the graph format. Despite intensive ongoing research in this area, domain-specific content, such as factory planning, requires time-consuming human assistance [8]. For this reason, the DKG approach will be further examined in this paper.

For the design of the DKG, it is advantageous to reuse existing ontologies. As shown in Figure 1, the DKG is responsible for representing the hierarchical structure and metadata of the imported documents. For this purpose, the *Document Components Ontology* (DoCO) can be used, which provides a structured vocabulary for document components [9]. The DoCO itself is based on the *Discourse Elements Ontology* (DEO) [10] and the *Documents Structural Patterns Ontology* (PO) [11]. These ontologies can be used to describe the structural elements of a document (e.g., chapters and sections), rhetorical elements (e.g., introductions and illustrations), and metadata (e.g., authors).

C. Remaining Challenges

Based on the related work presented, it can be said that RAG is a promising approach for knowledge management in the field of factory planning. The use of a DKG represents a promising approach for managing the knowledge base. However, there is no general approach for building this graph that can map both general requirements for RAG and domain-specific content. This paper aims to describe the structure and content of a DKG based on existing ontologies and its use for RAG in conjunction with another database.

III. Concept

This section introduces the concept of the DKG by detailing its design and structure. Following this, we demonstrate its application within a RAG architecture. To facilitate the development of the DKG, we divide it into two main areas: document structure and metadata management. The requirements for the document structure include:

- Mapping the hierarchical structure of the document.
- Annotating structural information on chunks, such as the predecessor.
- Storing the content and type of each chunk.

For metadata management, the requirements are:

- Providing a framework for managing domain-specific metadata within the DKG.
- Enabling simultaneous filtering across different categories.

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on February 09,2026 at 19:08:08 UTC from IEEE Xplore. Restrictions apply.

A. Design of the Document Knowledge Graph

This section describes the concepts for the structure of the DKG in the individual areas. The prefixes in the illustrations denote elements reused from existing ontologies. The central element of the DKG is the `dkg:document` class, to which individual documents belong. The prefix `dkg` indicates that this class was newly created as part of this work and is not derived from any existing ontology.

Management of Document Structure: Our approach to managing the document and content structure can be seen in Figure 2. Within the DKG, the structure is represented by the classes `dkg:content`, `doco:Chapter`, and `doco:Section`, while the last one can be nested within each other. The content of the document has been summarized using the class `doco:Elememt` and contains subclasses for the various types of content. The relation `dkg:hasPrevSection` is used to trace the sequence of the document content. This makes it possible to query previous and subsequent content for a specific `dkg:Element`.

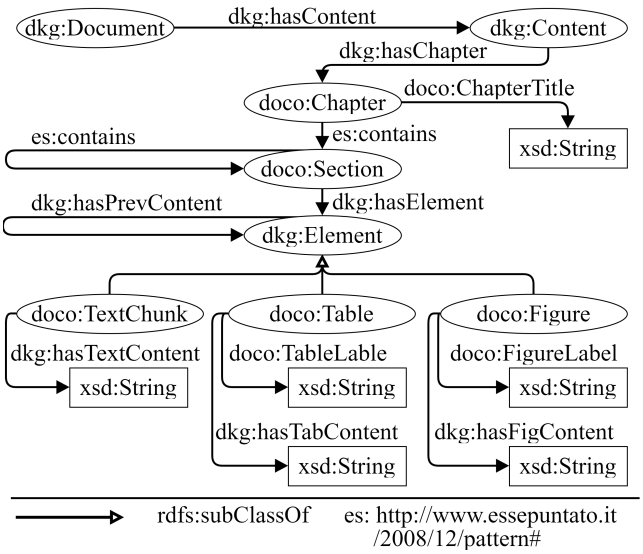

Fig. 2. DKG approach for document structure management.

Another crucial aspect of the DKG design is the naming of objects using *Uniform Resource Identifiers* (URI). This allows the document structure to be reflected in the URI, enhancing content traceability. Accordingly, for example, the third `doco:TextChunk` in chapter 2.4 of a document with ID 2 has the URI `Document2/Chapter2/Subchapter4/TextChunk3`.

Management of Meta-data: The specific management of meta-data varies significantly depending on the DKG's application domain. Figure 3 illustrates a general structure for meta-data management. Various independent meta-data can be linked to the `dkg:Document`. These are summarized in the class `dkg:MetaData`. The depicted *Subclasses 1-n* denote the individual categories used to classify the documents, which can also be grouped into classes. In the context of factory planning, categories such as the planning project, production line, or plant name can be employed to categorize and filter content. To group documents in the graph according to meta-data, classes are used instead of the literals suggested by DoCO.

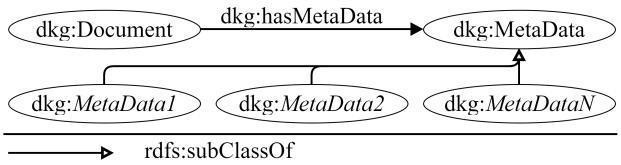

Fig. 3. DKG approach for meta-data management.

B. Integration of the Document Knowledge Graph in a Retrieval Augmented Generation Architecture

The concept of integrating the DKG into a RAG architecture to support another database (in this case, a vector database) is illustrated in Figure 4. he illustration includes both the core elements of the architecture and the phases of the RAG methodology, which are shown on the left side from top to bottom.

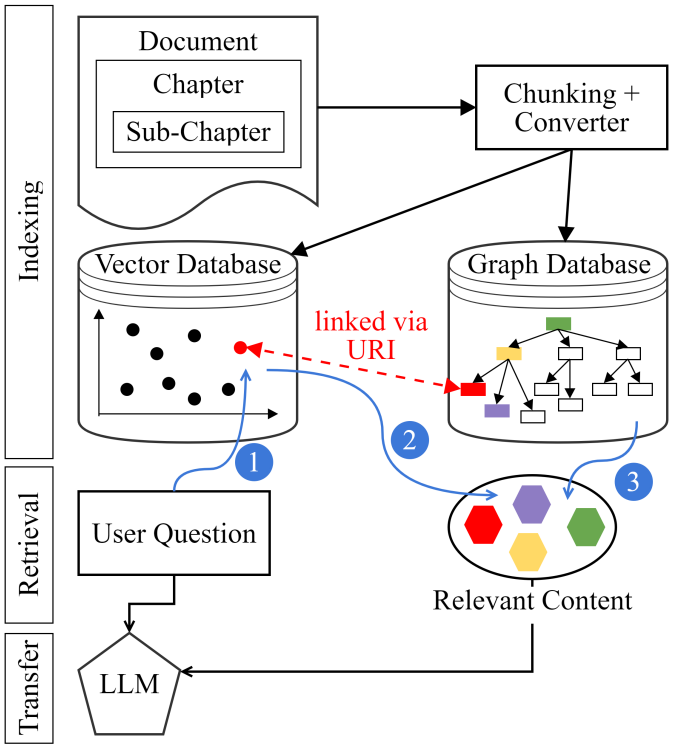

Fig. 4. Approach for RAG with DKG.

As shown in Figure 4, the process begins with the document in the indexing phase. The document must have a hierarchical structure, with chapters and sub-chapters that can be processed in a machine-readable manner. To add the documents to the databases, they are processed in the *Chunking + Converter* step. Here, the text sections are divided into individual chunks and assigned a URI according to the predefined logic. For

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on February 09,2026 at 19:08:08 UTC from IEEE Xplore. Restrictions apply.

import into the vector database, vector representations are created using a text embedding model, and these are added to the database with the URI as the ID. For the parallel inclusion of the content in the graph database to build the DKG, the document structure is converted into the presented format and supplemented with content such as chapter headings from the document. Additionally, desired meta-data categories are populated during the import.

In the RAG retrieval phase, a basic three-stage approach is depicted in blue in Figure 4. Starting from a user query, a search is made in the vector database using the query's embedding. The results of this search (indicated by the red dot in the vector database) are added in the second stage to the collection of chunks presumed to be relevant to the answer. For clarity, the number of results from the vector database in the illustration is limited to one. The expansion of the relevant content using the DKG in the third stage of retrieval can be carried out individually for each chunk retrieved from the vector database. To expand the relevant content with the DKG, the described linking of the databases via the URI is utilized. This allows a transition from the result of the vector search to the corresponding location in the DKG, shown in red within the figure. Here, the knowledge from the KG can be used in various ways. For example, further chunks from the document can be added to the relevant content by adding the following chunk, shown here in purple, to our red starting point via a query to the DKG Assuming that related contents of a document are also in a common (sub)-chapter, this additional context can provide the LLM with further knowledge about the content described in the red chunk. Another way to use the DKG is to supplement the relevant content with information available only through the graph structure. For instance, the LLM can be informed about the (sub)chapter heading (yellow node in the DKG) of the found chunk (red node). Or information about the entire document (e.g., meta-data, uploading user) can be provided via the green document node in the DKG. In this case, the information must be queried through predefined queries. To transfer the results to the LLM, they are integrated into predefined sentences to produce a natural language statement.

In the final RAG phase, the relevant content is transferred to the LLM along with the user question. With the information enriched by the DKG, the LLM can consider other knowledge such as chapter headings or meta-data when generating the answer. Additionally, follow-up queries from the user can include contact persons for the document, which could not be answered with a pure vector search alone.

Conclusion and Outlook

This paper presents a concept for extending classical vector-based RAG approaches by incorporating a KG. It also demonstrates how a DKG can be utilized to manage additional facts, such as metadata, and transfer them to the LLM as supplementary context information. Although the concept is still in its early phase, the initial results are promising and warrant further research.

In future work, we plan to quantitatively evaluate the performance of the DKG compared to a solely vector-based RAG approach, as presented in [4]. Additionally, we aim to increase the information density within the DKG. One approach to achieving this is the inclusion of cross-document links to further guide the RAG architecture. This can be done with the technique of keyword indexing, which enables chunks with related content to be linked via extracted keywords. Additional approaches, such as the consideration and summarization of text segments based on their embeddings, also offer potential for future research.

Moreover, we intend to expand the DKG with more comprehensive functions for managing the knowledge base. Creating links between documents could help structure the graph by connecting different versions or linking related documents based on domain-specific use cases. This enhanced structuring could improve the efficiency and effectiveness of knowledge management and retrieval systems.

Acknowledgment

The presented paper was produced as part of the research project MoFaPro. This project is funded by the AUDI AG. The present approach was developed within the research cluster "Digital Production" of the institute "AImotion Bavaria" at the Technische Hochschule Ingolstadt.

References

[1] F. Zhu, W. Lei, C. Wang, J. Zheng, S. Poria, and T.-S. Chua, "Retrieving and Reading: A Comprehensive Survey on Open-domain Question Answering," *arXiv preprint arXiv:2101.00774*, 2021.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[4] M. A. Khadija, A. Aziz, and W. Nurharjadmo, "Automating Information Retrieval from Faculty Guidelines: Designing a PDF-Driven Chatbot powered by OpenAI ChatGPT," in *2023 International Conference on Computer, Control, Informatics and its Applications (IC3INA)*. IEEE, 2023, pp. 394–399.

[5] O. Topsakal and T. C. Akinci, "Creating Large Language Model Applications Utilizing Langchain: A Primer on Developing LLM Apps Fast," in *International Conference on Applied Engineering and Natural Sciences*, vol. 1, no. 1, 2023, pp. 1050–1056.

[6] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–20, 2024.

[7] Y. Wang, N. Lipka, R. A. Rossi, A. Siu, R. Zhang, and T. Derr, "Knowledge Graph Prompting for Multi-Document Question Answering," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 206–19 214.

[8] A. Bytyci, L. Ramosaj, and E. Bytyci, "Review of automatic and semi-automatic creation of knowledge graphs from structured and unstructured data," in *RTA-CSIT*, 2023, pp. 72–79.

[9] A. Constantin, S. Peroni, S. Pettifer, D. Shotton, and F. Vitali, "The Document Components Ontology (DoCO)," *Semantic web*, vol. 7, no. 2, pp. 167–181, 2016.

[10] D. Shotton and S. Peroni, "The Discourse Elements Ontology (DEO)," 2015, Retrieved 12.04.2024. [Online]. Available: http://purl.org/spar/pso/2015-07-03

[11] F. V. Angelo Di Iorio and S. Peroni, "The Pattern Ontology: Describing documents by means of their structural components," 2013, Retrieved 12.04.2024. [Online]. Available: http://purl.org/spar/po

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on February 09,2026 at 19:08:08 UTC from IEEE Xplore. Restrictions apply.